

September 6, 2026 at 11:44

1. Introduction. Exercise 7.2.2.1–305 glues a small tilted domino over an ordinary one and gets ten two-layer pieces, the *windmill dominoes*. Its five parts ask for the arrangement whose upper layer looks like a windmill, for packings of the ten pieces into a 4×5 and a 2×10 box, and for the two ways of making the *upper* layer fill a rectangle instead. Exercise 306 then asks for the arrangements in which the twenty large squares and the twenty small ones each form a snake-in-the-box cycle, in the sense of exercise 172(b).

This program checks every count both answers print. The two exercises share a piece set and a coordinate system, so they share a program.

Answer 305 supplies the coordinates: the large square in row i and column j is the pair $(2i + 1)(2j + 1)$, and the small tilted square straddling two adjacent large squares is the midpoint between them. So a point with two odd coordinates is a large square, a point with exactly one is a small one, and a point with none is a corner of the large grid. Large squares have area 4, small ones area 2.

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strconv"
    "strings"
    "time"

    cells "github.com/sjnam/dancing-cells"
)
<Declarations 4>
<Functions 5>
func main() {
    <Read the command line 2>
    <Do what the mode asks 3>
}
```

2. The parts of exercise 305 are named as the exercise names them. *shapes* lists the distinct shapes the large squares make, which is how the picture in the answer to part (d) gets identified; *snake* is the whole of exercise 306.

<Read the command line 2> $\equiv$

```
mode := flag.String("mode", "pieces",
    "pieces, a, b, c, d, e8, e9, shapes, or snake")
verbose := flag.Bool("v", false, "list the options of each piece")
flag.Parse()
```

This code is used in section 1.

3. $\langle$ Do what the mode asks 3 $\rangle \equiv$
- ```
switch *mode {
 case "pieces":
 $\langle$ Check the piece set 20 $\rangle$
 case "shapes":
 $\langle$ List the shapes the large squares make 63 $\rangle$
 case "snake":
 $\langle$ Do exercise 306 91 $\rangle$
 default:
 report(*mode,*verbose)
}
```

This code is used in section 1.

**4. The windmill dominoes.** A point is a pair of integers; the parity of the pair says what kind of square sits there.

⟨Declarations 4⟩ ≡

```
type pt struct { a, b int }
```

This code is also defined in sections 7, 18, 19, 22, 26, 39, 44, 47, 77, 79, 80.

This code is used in section 1.

**5.** ⟨Functions 5⟩ ≡

```
func large(p pt) bool { return p.a % 2 ≠ 0 ∧ p.b % 2 ≠ 0 }
```

This code is also defined in sections 6, 8, 9, 10, 11, 12, 13, 14, 15, 16, 23, 24, 25, 27, 28, 31, 32, 36, 37, 38, 40, 41, 42, 43, 45, 46, 48, 50, 51, 52, 53, 54, 55, 57, 65, 67, 71, 73, 74, 75, 76, 78, 81, 84, 86, 87, 89, 93, 94, 95, 97, 98.

This code is used in section 1.

**6.** ⟨Functions 5⟩ +≡

```
func small(p pt) bool { return (p.a + p.b) % 2 ≠ 0 }
```

**7.** A piece is two large squares that share an edge and two small ones that share an edge. The exercise's picture shows that the small domino has to cover part of the large one, so at least one of its squares lies over a square of the large domino; the other may hang off.

⟨Declarations 4⟩ +≡

```
type piece struct {
 big [2]pt
 small [2]pt
}
```

**8.** A quarter turn about the origin keeps both kinds of square where they belong, because it preserves the parities.

⟨Functions 5⟩ +≡

```
func rot(p pt) pt { return pt{−p.b, p.a} }
```

**9.** ⟨Functions 5⟩ +≡

```
func (q piece) turn() piece {
 var r piece
 for i := 0; i < 2; i++ {
 r.big[i] = rot(q.big[i])
 r.small[i] = rot(q.small[i])
 }
 return r.tidy()
}
```

10. Sliding a piece by an even amount is the only translation that respects the parities. *tidy* slides it until it sits at the origin and puts each pair in order, so that two placements of the same piece look the same.

⟨ Functions 5 ⟩ +≡

```

func (q piece) tidy() piece {
 q = q.sorted()
 minA, minB := 1 << 30, 1 << 30
 for _, p := range q.cells() {
 if p.a < minA {
 minA = p.a
 }
 if p.b < minB {
 minB = p.b
 }
 }
 da, db := minA - minA & 1, minB - minB & 1
 for i := 0; i < 2; i++ {
 q.big[i] = pt{q.big[i].a - da, q.big[i].b - db}
 q.small[i] = pt{q.small[i].a - da, q.small[i].b - db}
 }
 return q
}

```

11. ⟨ Functions 5 ⟩ +≡

```

func (q piece) sorted() piece {
 if ahead(q.big[1], q.big[0]) {
 q.big[0], q.big[1] = q.big[1], q.big[0]
 }
 if ahead(q.small[1], q.small[0]) {
 q.small[0], q.small[1] = q.small[1], q.small[0]
 }
 return q
}

```

12. ⟨ Functions 5 ⟩ +≡

```

func ahead(u, v pt) bool { return u.a < v.a ∨ (u.a ≡ v.a ∧ u.b < v.b) }

```

13. ⟨ Functions 5 ⟩ +≡

```

func (q piece) cells() []pt {
 return []pt{q.big[0], q.big[1], q.small[0], q.small[1]}
}

```

14. Two pieces are the same when a turn and a slide carry one to the other; the exercise counts them that way, and reflections are *not* allowed, since a piece and its mirror image are different windmill dominoes. The canonical form is the least of the four turns.

⟨ Functions 5 ⟩ +≡

```
func (q piece) key() string {
 c := q.cells()
 sort.Slice(c, func(i, j int) bool { return ahead(c[i], c[j]) })
 s := ""
 for _, p := range c {
 s += fmt.Sprintf("%d,%d", p.a, p.b)
 }
 return s
}
```

15. ⟨ Functions 5 ⟩ +≡

```
func (q piece) canon() string {
 best, r := "", q.tidy()
 for i := 0; i < 4; i++ {
 if k := r.key(); best == "" ∨ k < best {
 best = k
 }
 r = r.turn()
 }
 return best
}
```

16. Building the set: take each of the two large dominoes, collect the small squares that touch either of its squares, pair each with a diagonal neighbour, and reduce. Exactly ten survive.

⟨ Functions 5 ⟩ +≡

```
func pieces() []piece {
 seen := map[string]piece{}
 for _, big := range [][]pt {{1, 1}, {1, 3}}, {{1, 1}, {3, 1}} {{
 ⟨ Glue every small domino onto this large one 17 ⟩
 }}
 var keys []string
 for k := range seen {
 keys = append(keys, k)
 }
 sort.Strings(keys)
 var out []piece
 for _, k := range keys {
 out = append(out, seen[k])
 }
 return out
}
```

17.  $\langle$  Glue every small domino onto this large one 17  $\rangle \equiv$

```

touching := map[pt]bool{ }
for _, g := range big {
 for _, d := range [pt]{ {-1, 0}, {1, 0}, {0, -1}, {0, 1} }{
 touching[pt{g.a + d.a, g.b + d.b}] = true
 }
}
for p := range touching {
 for _, d := range smallSteps {
 q := pt{p.a + d.a, p.b + d.b}
 if ¬small(q) {
 continue
 }
 pc := piece{big, [2]pt{p, q}}
 seen[pc.canon()] = pc.tidy()
 }
}

```

This code is used in section 16.

18. Two small squares share an edge when they step diagonally by one; two large squares share an edge when they step by two. These two step sets come up everywhere, and exercise 306 needs them by name.

$\langle$  Declarations 4  $\rangle + \equiv$

```
var smallSteps = [pt]{ {-1, -1}, {-1, 1}, {1, -1}, {1, 1} }
```

19.  $\langle$  Declarations 4  $\rangle + \equiv$

```
var bigSteps = [pt]{ {-2, 0}, {2, 0}, {0, -2}, {0, 2} }
```

20. Answer 305 lists the four placements of “the leftmost piece”:  $\{13, 15, 12, 23\}$ ,  $\{33, 53, 23, 32\}$ ,  $\{33, 31, 34, 23\}$  and  $\{31, 11, 41, 32\}$ . They should be the four turns of a single one of the ten.

$\langle$  Check the piece set 20  $\rangle \equiv$

```

ps := pieces()
fmt.Printf("windmill dominoes: %d\n", len(ps))
for i, q := range ps {
 fmt.Printf("%d: %d large %v, %d small %v\n", i,
 q.big[0], q.big[1], q.small[0], q.small[1])
}
 $\langle$ Compare with the placements answer 305 prints 21 $\rangle$

```

This code is used in section 3.

21.  $\langle \text{Compare with the placements answer 305 prints 21} \rangle \equiv$

```

printed := [][4]pt{
 {{1, 3}, {1, 5}, {1, 2}, {2, 3}},
 {{3, 3}, {5, 3}, {2, 3}, {3, 2}},
 {{3, 3}, {3, 1}, {3, 4}, {2, 3}},
 {{3, 1}, {1, 1}, {4, 1}, {3, 2}},
}
keys := map[string]bool{ }
for _, pl := range printed {
 keys[piece{[2]pt{pl[0], pl[1]}, [2]pt{pl[2], pl[3]}}.canon()] = true
}
known := map[string]bool{ }
for _, q := range ps {
 known[q.canon()] = true
}
fmt.Printf("the four placements answer 305 prints are one piece: %v\n",
 len(keys) == 1)
for k := range keys {
 fmt.Printf("and it is one of the ten: %v\n", known[k])
}

```

This code is used in section 20.

**22. Boxes and layers.** A box of  $m$  rows and  $n$  columns holds large squares at the odd-odd points strictly inside  $2m$  by  $2n$ . The small squares of a piece may or may not have to stay inside, and which of the two layers has to be filled exactly changes from part to part; that is the whole of the difference between (a)–(c) and (d)–(e).

⟨Declarations 4⟩ +≡

```
type box struct { m, n int }
```

**23.** ⟨Functions 5⟩ +≡

```
func (x box) insideLarge(p pt) bool {
 return large(p) $\wedge$ $0 < p.a \wedge p.a < 2 * x.m \wedge 0 < p.b \wedge p.b < 2 * x.n$
}
```

**24.** Item names have to survive the parser, and a coordinate can go negative, so everything is shifted by eight before it is written down.

⟨Functions 5⟩ +≡

```
func sq(p pt) string { return fmt.Sprintf("s%d.%d", $p.a + 8, p.b + 8$) }
```

**25.** ⟨Functions 5⟩ +≡

```
func unsq(s string) pt {
 $i := strings.IndexByte(s, '')$
 $a, _ := strconv.Atoi(s[1:i])$
 $b, _ := strconv.Atoi(s[i + 1:])$
 return pt{ $a - 8, b - 8$ }
}
```

**26.** A problem is a box together with a demand on the upper layer. When *pairs* is set, a piece may only be placed so that its small domino is one of the listed pairs; that is how part (a) asks for a windmill. When *smallPrim* is set, those small squares are primary and have to be filled exactly, which is what parts (d) and (e) want. When neither is set the small squares merely have to keep out of each other's way inside the box.

⟨Declarations 4⟩ +≡

```
type prob struct {
 x box
 smallPrim map[pt]bool
 pairs map[[2]pt]bool
 piecesPrim bool
 largePrim bool
}
```

**27.** ⟨Functions 5⟩ +≡

```
func pairKey(u, v pt) [2]pt {
 if ahead(v, u) {
 $u, v = v, u$
 }
 return [2]pt{ u, v }
}
```



28.  $\langle \text{Functions 5} \rangle + \equiv$

```

func (w prob) options(q piece) []piece {
 var out []piece
 seen := map[piece]bool{ }
 r := q.tidy()
 for turn := 0; turn < 4; turn ++ {
 for da := -4 * w.x.m; da ≤ 4 * w.x.m; da += 2 {
 for db := -4 * w.x.n; db ≤ 4 * w.x.n; db += 2 {
 $\langle \text{Slide the piece there and keep it if it fits 29} \rangle$
 }
 }
 r = r.turn()
 }
 return out
}

```

29.  $\langle \text{Slide the piece there and keep it if it fits 29} \rangle \equiv$

```

var s piece
for i := 0; i < 2; i ++ {
 s.big[i] = pt{r.big[i].a + da, r.big[i].b + db}
 s.small[i] = pt{r.small[i].a + da, r.small[i].b + db}
}
if $\neg w.x.insideLarge(s.big[0]) \vee \neg w.x.insideLarge(s.big[1])$ {
 continue
}
 $\langle \text{Ask whether the upper layer allows this placement 30} \rangle$
if s = s.sorted(); $\neg seen[s]$ {
 seen[s] = true
 out = append(out, s)
}

```

This code is used in section 28.

30.  $\langle \text{Ask whether the upper layer allows this placement 30} \rangle \equiv$

```

switch {
case w.pairs ≠ Λ :
 if $\neg w.pairs[pairKey(s.small[0], s.small[1])]$ {
 continue
 }
case w.smallPrim ≠ Λ :
 if $\neg w.smallPrim[s.small[0]] \vee \neg w.smallPrim[s.small[1]]$ {
 continue
 }
default:
 if outside(s.small[0], w.x) $\vee outside(s.small[1], w.x)$ {
 continue
 }
}

```

This code is used in section 29.

31.  $\langle \text{Functions } 5 \rangle + \equiv$

```
func outside(p pt, x box) bool {
 return p.a ≤ 0 ∨ p.a ≥ 2 * x.m ∨ p.b ≤ 0 ∨ p.b ≥ 2 * x.n
}
```

**32. The exact cover problem.** Each piece is an item, primary when it must be used and secondary when it need not be; each square of a layer that has to be filled exactly is a primary item; each square that merely has to stay unshared is a secondary item.

⟨ Functions 5 ⟩ +≡

```

func (w prob) build()(string, [][]piece) {
 ps := pieces()
 all := make([][]piece, len(ps))
 var prim, sec []string
 for i := range ps {
 all[i] = w.options(ps[i])
 ⟨ Declare the item for piece i 33 ⟩
 }
 ⟨ Declare the items for the squares 34 ⟩
 var b strings.Builder
 b.WriteString(strings.Join(prim, "□"))
 if len(sec) > 0 {
 b.WriteString("□|□" + strings.Join(sec, "□"))
 }
 b.WriteByte('\\n')
 ⟨ Write one option per placement 35 ⟩
 return b.String(), all
}

```

**33.** ⟨ Declare the item for piece *i* 33 ⟩ ≡

```

if w.piecesPrim {
 prim = append(prim, fmt.Sprintf("P%d", i))
} else {
 sec = append(sec, fmt.Sprintf("P%d", i))
}

```

This code is used in section 32.

**34.** A small square that is neither wanted nor forbidden gets no item at all: when *smallPrim* is set, the placements were already restricted to it.

⟨ Declare the items for the squares 34 ⟩ ≡

```

for a := 1; a < 2 * w.x.m; a ++ {
 for b := 1; b < 2 * w.x.n; b ++ {
 p := pt{a, b}
 switch {
 case large(p):
 if w.largePrim {
 prim = append(prim, sq(p))
 } else {
 sec = append(sec, sq(p))
 }
 case small(p):
 if w.smallPrim[p] {
 prim = append(prim, sq(p))
 } else if w.smallPrim ≡ Λ {
 sec = append(sec, sq(p))
 }
 }
 }
}

```

This code is used in section 32.

**35.** ⟨ Write one option per placement 35 ⟩ ≡

```

for i := range ps {
 for s := range all[i] {
 fmt.Fprintf(&b, "P%d_%s_%s_%s\n", i,
 sq(s.big[0]), sq(s.big[1]), sq(s.small[0]), sq(s.small[1]))
 }
}

```

This code is used in section 32.

**36.** A solution arrives as a list of options, each a list of item names; turning it back into placements is a matter of parities.

⟨ Functions 5 ⟩ +≡

```

func parse(opt []cells.Option) []piece {
 var out []piece
 for _, o := range opt {
 var q piece
 nb, ns := 0, 0
 for _, t := range o[1:] {
 if p := unsq(t); large(p) {
 q.big[nb] = p
 nb++
 } else {
 q.small[ns] = p
 ns++
 }
 }
 out = append(out, q.sorted())
 }
 return out
}

```

**37.** ⟨ Functions 5 ⟩ +≡

```

func bigs(sol []piece) []pt {
 var out []pt
 for _, q := range sol {
 out = append(out, q.big[0], q.big[1])
 }
 return out
}

```

**38.** ⟨ Functions 5 ⟩ +≡

```

func smalls(sol []piece) []pt {
 var out []pt
 for _, q := range sol {
 out = append(out, q.small[0], q.small[1])
 }
 return out
}

```

**39. Symmetries of a box.** Two arrangements in the same box are the same puzzle when one of the box's own symmetries carries the other to it: four of them for an oblong, eight for a square. Which of them actually apply depends on the problem—the windmill of part (a) is chiral, so its reflections drop out—so a symmetry counts only when it carries every legal placement to a legal one.

⟨Declarations 4⟩ +≡

```
type xform struct {
 name string
 f func(pt) pt
}
```

40. ⟨Functions 5⟩ +≡

```
func (x box) group() []xform {
 m, n := 2 * x.m, 2 * x.n
 g := []xform{
 {"id", func(p pt) pt { return p }},
 {"flip_rows", func(p pt) pt { return pt{m - p.a, p.b} }},
 {"flip_cols", func(p pt) pt { return pt{p.a, n - p.b} }},
 {"turn_180", func(p pt) pt { return pt{m - p.a, n - p.b} }},
 }
 if x.m == x.n {
 g = append(g,
 xform{"transpose", func(p pt) pt { return pt{p.b, p.a} }},
 xform{"turn_90", func(p pt) pt { return pt{p.b, m - p.a} }},
 xform{"turn_270", func(p pt) pt { return pt{n - p.b, p.a} }},
 xform{"anti-transpose", func(p pt) pt { return pt{n - p.b, m - p.a} }}
)
 }
 return g
}
```

41. ⟨Functions 5⟩ +≡

```
func (t xform) on(q piece) piece {
 var r piece
 for i := 0; i < 2; i++ {
 r.big[i] = t.f(q.big[i])
 r.small[i] = t.f(q.small[i])
 }
 return r.sorted()
}
```

42. Because a placement determines its piece, a symmetry that carries every legal placement to a legal one automatically permutes the ten pieces, and so carries solutions to solutions.

⟨ Functions 5 ⟩ +≡

```

func (w prob) symmetries(all [][]piece) []xform {
 legal := map[piece]bool{ }
 for _, l := range all {
 for _, q := range l {
 legal[q] = true
 }
 }
 var out []xform
 for _, t := range w.x.group() {
 ok := true
 for q := range legal {
 if ¬legal[t.on(q)] {
 ok = false
 break
 }
 }
 if ok {
 out = append(out, t)
 }
 }
 return out
}

```

43. ⟨ Functions 5 ⟩ +≡

```

func skey(sol []piece) string {
 c := append([]piece(Λ), sol...)
 sort.Slice(c, func(i, j int) bool {
 if c[i].big[0] ≠ c[j].big[0] {
 return ahead(c[i].big[0], c[j].big[0])
 }
 return ahead(c[i].small[0], c[j].small[0])
 })
 var b strings.Builder
 for _, q := range c {
 fmt.Fprintf(&b, "%v%v%v%v;", q.big[0], q.big[1], q.small[0], q.small[1])
 }
 return b.String()
}

```

44. A class of arrangements is an orbit of the group. Counting classes by dividing the number of arrangements by the group order is wrong whenever some arrangement is symmetric, so each one contributes the order of its own stabilizer instead: a class whose stabilizer has order  $f$  holds  $G/f$  arrangements, so it earns  $f$  towards a total that is divided by  $G$  at the end. Answer 305 splits its counts exactly this way, writing  $501484 = 2 \cdot 4 + 4 \cdot 125369$ .

⟨ Declarations 4 ⟩ +≡

```

type tally struct { sols, weight int }

```

45.  $\langle \text{Functions 5} \rangle + \equiv$

```
func (t *tally) add(fix int) { t.sols++; t.weight += fix }
```

46.  $\langle \text{Functions 5} \rangle + \equiv$

```
func (t tally) classes(g int) int { return t.weight/g }
```



**47. The shape of a layer.** Answer 305 speaks of arrangements “whose small squares do at least form a symmetric shape”. A shape is symmetric when a reflection or turn about its own centre carries it to itself; where it happens to sit is beside the point. Only a shape as wide as it is tall can survive a quarter turn or a diagonal reflection, so those are asked about only then.

⟨Declarations 4⟩ +≡

```
var shapeMaps = []struct {
 name string
 f func(pt, int, int) pt
 sq bool
}{
 {"mirror_across_a_row", func(p pt, ha, hb int) pt { return pt{ha - p.a, p.b} }, false},
 {"mirror_across_a_column", func(p pt, ha, hb int) pt { return pt{p.a, hb - p.b} }, false},
 {"turn_180", func(p pt, ha, hb int) pt { return pt{ha - p.a, hb - p.b} }, false},
 {"mirror_across_a_diagonal", func(p pt, ha, hb int) pt { return pt{p.b, p.a} }, true},
 {"mirror_across_the_other_diagonal", func(p pt, ha, hb int) pt { return pt{hb - p.b, ha -
 p.a} }, true},
 {"turn_90", func(p pt, ha, hb int) pt { return pt{p.b, ha - p.a} }, true},
 {"turn_270", func(p pt, ha, hb int) pt { return pt{hb - p.b, p.a} }, true},
}
```

48. ⟨Functions 5⟩ +≡

```
func shapeKind(ps []pt) string {
 set := map[pt]bool{}
 for _, p := range ps {
 set[p] = true
 }
 loA, hiA, loB, hiB := span(ps)
 ha, hb := loA + hiA, loB + hiB
 var got []string
 for _, t := range shapeMaps {
 ⟨Try this map on the shape 49⟩
 }
 return strings.Join(got, ", ")
}
```

49. ⟨Try this map on the shape 49⟩ ≡

```
if t.sq ∧ hiA - loA ≠ hiB - loB {
 continue
}
ok := true
for p := range set {
 if ¬set[t.f(p, ha, hb)] {
 ok = false
 break
 }
}
if ok {
 got = append(got, t.name)
}
```

This code is used in section 48.

50.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func span(ps []pt)(loA, hiA, loB, hiB int) {
 loA, hiA, loB, hiB = 1 << 30, -(1 << 30), 1 << 30, -(1 << 30)
 for _, p := range ps {
 if p.a < loA {
 loA = p.a
 }
 if p.a > hiA {
 hiA = p.a
 }
 if p.b < loB {
 loB = p.b
 }
 if p.b > hiB {
 hiB = p.b
 }
 }
 return
}

```

51. There is a narrower question one can ask instead: is the shape symmetric about the horizontal or the vertical axis *of the box*? That is what one sees when the output is drawn inside a fixed frame, and it turns out to be what answer 305 counts in part (a).

$\langle \text{Functions } 5 \rangle + \equiv$

```

func (w prob) frameSym(ps []pt) bool {
 set := map[pt]bool{ }
 for _, p := range ps {
 set[p] = true
 }
 for _, t := range w.x.group() [1:3] {
 ok := true
 for p := range set {
 if $\neg \text{set}[t.f(p)]$ {
 ok = false
 break
 }
 }
 if ok {
 return true
 }
 }
 return false
}

```

52.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func spans(ps []pt, m, n int) bool {
 loA, hiA, loB, hiB := span(ps)
 return (hiA - loA)/2 + 1 ≤ m ∧ (hiB - loB)/2 + 1 ≤ n
}

```

53.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func grid(x box, sol []piece) string {
 on := map[pt]bool{ }
 for _, p := range bigs(sol) {
 on[p] = true
 }
 s := ""
 for a := 1; a < 2 * x.m; a += 2 {
 s += "UUUUUUU"
 for b := 1; b < 2 * x.n; b += 2 {
 if on[pt{a, b}] {
 s += "#"
 } else {
 s += "."
 }
 }
 s += "\n"
 }
 return s
}

```

**54. The five parts of exercise 305.** Part (a) uses a  $5 \times 5$  box and asks that the small squares of every option be one of  $\{34, 45\}$ ,  $\{47, 56\}$ ,  $\{76, 65\}$  or  $\{63, 54\}$  — the four blades of a windmill turning about the middle of the box. Only four pieces are used, so the piece items are secondary; the eight small squares are the primary ones, and the large squares merely have to avoid each other.

Parts (b) and (c) fill a box with all ten pieces: large squares primary, small squares secondary and confined to the box.

Parts (d) and (e) turn that upside down. The upper layer has to fill a tilted rectangle exactly, so those twenty small squares are primary; the large squares only have to keep out of each other's way, inside a box big enough to hold them.

⟨ Functions 5 ⟩ +≡

```
func rect(c pt, i, j int) map[pt]bool {
 out := map[pt]bool{}
 for u := 0; u ≤ i; u++ {
 for v := 0; v ≤ j; v++ {
 out[pt{c.a + u + v, c.b - u + v}] = true
 }
 }
 return out
}
```

**55.** ⟨ Functions 5 ⟩ +≡

```
func part(which string) prob {
 switch which {
 case "a":
 ⟨ Set up the windmill of part (a) 56 ⟩
 case "b":
 return prob{x: box{4, 5}, piecesPrim: true, largePrim: true}
 case "c":
 return prob{x: box{2, 10}, piecesPrim: true, largePrim: true}
 case "d":
 return prob{x: box{7, 7}, smallPrim: rect(pt{4, 7}, 3, 4), piecesPrim: true}
 case "e8":
 return prob{x: box{8, 8}, smallPrim: rect(pt{3, 4}, 1, 9), piecesPrim: true}
 case "e9":
 return prob{x: box{9, 9}, smallPrim: rect(pt{4, 5}, 1, 9), piecesPrim: true}
 }
 panic("no such part")
}
```

**56.** ⟨ Set up the windmill of part (a) 56 ⟩ ≡

```
w := prob{x: box{5, 5}, smallPrim: map[pt]bool{ }, pairs: map[[2]pt]bool{ }}
for _, q := range [[2]pt] {
 {{3, 4}, {4, 5}}, {{4, 7}, {5, 6}}, {{7, 6}, {6, 5}}, {{6, 3}, {5, 4}},
}{
 w.pairs[pairKey(q[0], q[1])] = true
 w.smallPrim[q[0]] = true
 w.smallPrim[q[1]] = true
}
return w
```

This code is used in section 55.

57.  $\langle \text{Functions 5} \rangle + \equiv$

```
func report(which string, verbose bool) {
 w := part(which)
 in, all := w.build()
 $\langle \text{Say how big the problem is 58} \rangle$
 sym := w.symmetries(all)
 g := len(sym)
 $\langle \text{Count the solutions and sort them by symmetry 59} \rangle$
 $\langle \text{Say what the counts came to 62} \rangle$
}
```

58.  $\langle \text{Say how big the problem is 58} \rangle \equiv$

```
nopt := 0
for i, l := range all {
 nopt += len(l)
 if verbose {
 fmt.Printf("%%piece%%d:%%d%%options\n", i, len(l))
 }
}
sym0 := w.symmetries(all)
var names []string
for _, t := range sym0 {
 names = append(names, t.name)
}
fmt.Printf("part(%%s):%%dx%%d%%box,%%d%%options,%%symmetries%%v\n",
 which, w.x.m, w.x.n, nopt, names)
```

This code is used in section 57.

59.  $\langle \text{Count the solutions and sort them by symmetry 59} \rangle \equiv$

```
t0 := time.Now()
xc := cells.NewXCC()
total := 0
stab := map[int]int{ }
var bigSym, bigMirror, smallSym, smallAsym, bigFrame, fitting tally
shapes := map[string]int{ }
for s := range xc.Dance(strings.NewReader(in)).Solutions {
 sol := parse(s)
 total ++
 $\langle \text{Work out the stabilizer of this arrangement 60} \rangle$
 $\langle \text{Tally what its two layers look like 61} \rangle$
}
el := time.Since(t0).Round(time.Millisecond)
```

This code is used in section 57.

60.  $\langle$  Work out the stabilizer of this arrangement 60  $\rangle \equiv$

```

k := skey(sol)
fix := 0
for _, t := range sym {
 var moved []piece
 for _, q := range sol {
 moved = append(moved, t.on(q))
 }
 if skey(moved) $\equiv$ k {
 fix ++
 }
}
stab[fix]++

```

This code is used in section 59.

61.  $\langle$  Tally what its two layers look like 61  $\rangle \equiv$

```

if bk := shapeKind(bigs(sol)); bk $\neq$ "" {
 bigSym.add(fix)
 shapes[bk]++
 if strings.Contains(bk, "mirror") {
 bigMirror.add(fix)
 }
}
if w.frameSym(bigs(sol)) {
 bigFrame.add(fix)
}
if shapeKind(smalls(sol)) $\neq$ "" {
 smallSym.add(fix)
 if fix $\equiv$ 1 {
 smallAsym.add(fix)
 }
}
if spans(bigs(sol), 5, 5) {
 fitting.add(fix)
}

```

This code is used in section 59.

62.  $\langle$  Say what the counts came to 62  $\rangle \equiv$

```
fmt.Printf("%d solutions, %d nodes, %s\n", total, xc.Nodes(), el)
for f := 1; f ≤ g; f++ {
 if stab[f] > 0 {
 fmt.Printf("stabilizer of order %d: %d arrangements = %d classes of %d\n",
 f, stab[f], stab[f] * f/g, g/f)
 }
}
fmt.Printf("large squares form a symmetric shape: %d arrangements, %d classes" +
 "%d with a mirror, %d about an axis of the box)\n",
 bigSym.sols, bigSym.classes(g), bigMirror.classes(g), bigFrame.classes(g))
fmt.Printf("small squares form a symmetric shape: %d arrangements, %d classes" +
 "%d of them asymmetric)\n",
 smallSym.sols, smallSym.classes(g), smallAsym.classes(g))
if which ≡ "d" {
 fmt.Printf("large squares fit a 5x5 box: %d arrangements, %d classes\n",
 fitting.sols, fitting.classes(g))
}
for k, v := range shapes {
 fmt.Printf("large shape fixed by %s: %d arrangements\n", k, v)
}
```

This code is used in section 57.

63. Answer 305 says that 2·3 of the arrangements of part (d) “have large squares that form the symmetric shape shown”. To find which shape that is, one has to count the arrangements per shape, and the picture in the book has to be read.

$\langle$  List the shapes the large squares make 63  $\rangle \equiv$

```
w := part("d")
in, _ := w.build()
xc := cells.NewXCC()
count := map[string]int{}
kind := map[string]string{}
for s := range xc.Dance(strings.NewReader(in)).Solutions {
 sol := parse(s)
 g := trim(grid(w.x, sol))
 count[g]++
 kind[g] = shapeKind(bigs(sol))
}
 $\langle$ Print the shapes that are symmetric 64 $\rangle$
```

This code is used in section 3.

64.  $\langle$  Print the shapes that are symmetric 64  $\rangle \equiv$

```

var keys []string
for k := range count {
 if kind[k] != "" {
 keys = append(keys, k)
 }
}
sort.Strings(keys)
fmt.Printf("%d distinct shapes in all, %d of them symmetric\n",
 len(count), len(keys))
for _, k := range keys {
 fmt.Printf("%d arrangements [%s]: %d\n", count[k], kind[k], k)
}

```

This code is used in section 63.

65.  $\langle$  Functions 5  $\rangle + \equiv$

```

func trim(g string) string {
 lines := strings.Split(strings.TrimRight(g, "\n"), "\n")
 for len(lines) > 0 ^ strings.Trim(lines[0], ".") == "" {
 lines = lines[1:]
 }
 for len(lines) > 0 ^ strings.Trim(lines[len(lines)-1], ".") == "" {
 lines = lines[:len(lines)-1]
 }
 return strings.Join(lines, "\n") + "\n"
}

```



**66. Two snakes at once.** Exercise 306 asks in how many ways the ten windmill dominoes can be arranged so that the twenty large squares define a snake-in-the-box cycle and so do the twenty small ones. A snake-in-the-box cycle, by exercise 172(b), is a set of vertices whose induced subgraph is a single cycle: every chosen vertex has exactly two chosen neighbours, and the whole is connected.

Answer 306 writes this as one big problem for Algorithm M, with an item that decides each square, an item that gives it two neighbours, and items of multiplicity  $[0..3]$  to forbid a bare 4-cycle; it warns that nonsharp branching is needed. I could not get that to finish, so this program takes the structure apart instead.

The key is that an induced cycle has no chords. So two large squares that share an edge and both lie on the cycle are neighbours *along* the cycle, and the ten large dominoes are therefore a perfect matching of a 20-cycle—of which there are exactly two. The same argument applies to the upper layer. That turns the problem into: choose the cycle, choose one of its two matchings, and hang a small domino on each large one.

**67.** Choosing the cycle is itself an exact cover problem, and a small one. Each cell of the box is either empty or has exactly two chosen neighbours; an item of multiplicity  $[20..20]$  counts the chosen ones; and the four cells around a lattice point carry an item of multiplicity  $[0..3]$ , which is answer 306's own device for forbidding a 4-cycle. Connectivity an exact cover cannot say, so it is checked afterwards.

⟨ Functions 5 ⟩ +≡

```
func cycleModel(x box, want int) string {
 var prim, sec []string
 var b strings.Builder
 ⟨ Name the items of the cycle problem 68 ⟩
 b.WriteString(strings.Join(prim, "□") + "□|□" + strings.Join(sec, "□") + "\n")
 for a := 1; a < 2 * x.m; a += 2 {
 for c := 1; c < 2 * x.n; c += 2 {
 ⟨ Write the options for cell (a,c) 69 ⟩
 }
 }
 return b.String()
}
```

**68.** ⟨ Name the items of the cycle problem 68 ⟩ ≡

```
for a := 1; a < 2 * x.m; a += 2 {
 for c := 1; c < 2 * x.n; c += 2 {
 prim = append(prim, "c" + sq(pt{a, c}))
 sec = append(sec, "v" + sq(pt{a, c}))
 }
}
for a := 2; a < 2 * x.m - 1; a += 2 {
 for c := 2; c < 2 * x.n - 1; c += 2 {
 prim = append(prim, "0:3|B" + sq(pt{a, c}))
 }
}
prim = append(prim, fmt.Sprintf("%d|N", want))
```

This code is used in section 67.

69.  $\langle$  Write the options for cell (a,c) 69  $\rangle \equiv$

```

p := pt{a, c}
fmt.Fprintf(&b, "c%s_v%s:0\n", sq(p), sq(p))
var nb []pt
for _, d := range bigSteps {
 if q := (pt{p.a + d.a, p.b + d.b}); x.insideLarge(q) {
 nb = append(nb, q)
 }
}
var caps string
for _, d := range []pt {{-1, -1}, {-1, 1}, {1, -1}, {1, 1}} {
 q := pt{p.a + d.a, p.b + d.b}
 if 1 < q.a & q.a < 2 * x.m - 1 & 1 < q.b & q.b < 2 * x.n - 1 {
 caps += "B" + sq(q)
 }
}
 $\langle$ Write one option per choice of two neighbours 70 $\rangle$

```

This code is used in section 67.

70.  $\langle$  Write one option per choice of two neighbours 70  $\rangle \equiv$

```

for i := 0; i < len(nb); i++ {
 for j := i + 1; j < len(nb); j++ {
 fmt.Fprintf(&b, "c%s_v%s:1_N", sq(p), sq(p))
 for k, q := range nb {
 col := "0"
 if k == i || k == j {
 col = "1"
 }
 fmt.Fprintf(&b, "v%s:%s", sq(q), col)
 }
 b.WriteString(caps + "\n")
 }
}

```

This code is used in section 69.

71.  $\langle \text{Functions 5} \rangle + \equiv$

```

func snakeCycles(x box, want int) [[]]pt {
 mc := cells.NewMCC()
 var out [[]]pt
 for s := range mc.Dance(strings.NewReader(cycleModel(x, want))).Solutions {
 var on [pt]
 for _, o := range s {
 for _, t := range o {
 if strings.HasPrefix(t, "v") $\wedge$ strings.HasSuffix(t, ":1") {
 on = append(on, unsq(t[1:len(t) - 2]))
 }
 }
 }
 $\langle \text{Keep this one if it is a single cycle filling the box 72} \rangle$
 }
 return out
}

```

72. The cells come back once per option that mentions them, so they need weeding; and the bounding box has to be the whole box, or the same shape would be found again in a larger one.

$\langle \text{Keep this one if it is a single cycle filling the box 72} \rangle \equiv$

```

on = unique(on)
if len(on) $\neq$ want $\vee$ $\neg$ oneComponent(on, bigSteps) {
 continue
}
loA, hiA, loB, hiB := span(on)
if loA $\neq$ 1 $\vee$ hiA $\neq$ 2 * x.m - 1 $\vee$ loB $\neq$ 1 $\vee$ hiB $\neq$ 2 * x.n - 1 {
 continue
}
out = append(out, on)

```

This code is used in section 71.

73.  $\langle \text{Functions 5} \rangle + \equiv$

```

func unique(ps [pt]) [pt] {
 seen := map[pt]bool{ }
 var out [pt]
 for _, p := range ps {
 if $\neg$ seen[p] {
 seen[p] = true
 out = append(out, p)
 }
 }
 sort.Slice(out, func(i, j int) bool { return ahead(out[i], out[j]) })
 return out
}

```

74.  $\langle \text{Functions } 5 \rangle + \equiv$ 

```

func oneComponent(ps []pt, steps []pt) bool {
 set := map[pt]bool{ }
 for $_, p$:= range ps {
 set[p] = true
 }
 seen := map[pt]bool{ps[0]: true}
 queue := []pt{ps[0]}
 for len(queue) > 0 {
 p := queue[0]
 queue = queue[1:]
 for $_, d$:= range steps {
 if $q := (\mathbf{pt}\{p.a + d.a, p.b + d.b\}); \text{set}[q] \wedge \neg \text{seen}[q]$ {
 seen[q] = true
 queue = append(queue, q)
 }
 }
 }
 return len(seen) $\equiv$ len(set)
}

```

**75. Perfect matchings.** Walking round a cycle is easy when every vertex has exactly two neighbours in it: step to whichever neighbour has not been visited.

(Functions 5) +≡

```

func ringOf(ps []pt, steps []pt) []pt {
 set := map[pt]bool{ }
 for _, p := range ps {
 set[p] = true
 }
 out, seen := []pt{ps[0]}, map[pt]bool{ps[0]: true}
 for len(out) < len(ps) {
 cur, moved := out[len(out) - 1], false
 for _, d := range steps {
 if q := (pt{cur.a + d.a, cur.b + d.b}); set[q] ∧ ¬seen[q] {
 seen[q] = true
 out = append(out, q)
 moved = true
 break
 }
 }
 if ¬moved {
 return out
 }
 }
 return out
}

```

**76.** A cycle of even length has two perfect matchings: take every other edge, starting at one end of an edge or at the other.

(Functions 5) +≡

```

func matchings(ps []pt) [][][2]pt {
 ring := ringOf(ps, bigSteps)
 var out [][][2]pt
 for off := 0; off < 2; off ++ {
 var m [][2]pt
 for k := off; k < off + len(ring); k += 2 {
 u, v := ring[k % len(ring)], ring[(k + 1) % len(ring)]
 if ahead(v, u) {
 u, v = v, u
 }
 m = append(m, [2]pt{u, v})
 }
 out = append(out, m)
 }
 return out
}

```

77. An attachment says which of the ten pieces a large domino becomes when a small domino is glued to it, and where that small domino lies relative to the domino's first square. The step from the first large square to the second is the only thing that tells the two orientations apart, so the attachments are indexed by it: twenty for each.

⟨Declarations 4⟩ +≡

```
type attach struct {
 piece int
 small [2]pt
}
```

78. ⟨Functions 5⟩ +≡

```
func glue() map[pt][attach] {
 out := map[pt][attach]{}
 for i, q := range pieces() {
 r := q.tidy()
 for turn := 0; turn < 4; turn++ {
 s := r.sorted()
 d := pt{s.big[1].a - s.big[0].a, s.big[1].b - s.big[0].b}
 out[d] = append(out[d], attach{i, [2]pt{
 {s.small[0].a - s.big[0].a, s.small[0].b - s.big[0].b},
 {s.small[1].a - s.big[0].a, s.small[1].b - s.big[0].b},
 }})
 r = r.turn()
 }
 }
 return out
}
```

**79. Walking the dominoes.** With the lower layer settled, what is left is to give each large domino a small one: ten choices out of twenty apiece, with the ten pieces all different and the upper layer growing into a cycle. Two prunings make this quick. A small square may never have three neighbours. And a square that no later domino can reach will never gain another neighbour, so it must have two already; a cycle that closes before all twenty squares are down can never take in the rest.

⟨Declarations 4⟩ +≡

```

type walk struct {
 dom []2pt
 gl map[pt][attach]
 reach []map[pt]bool
 used [10]bool
 deg map[pt]int
 edges int
 small []pt
 sol []piece
 found [][]piece
 loose bool
}

```

**80.** ⟨Declarations 4⟩ +≡

```

type edge struct { u, v pt }

```

**81.** ⟨Functions 5⟩ +≡

```

func (w *walk) run(k int) {
 if k ≡ len(w.dom) {
 ⟨Keep the arrangement if the upper layer will do 82⟩
 return
 }
 d := w.dom[k]
 step := pt{d[1].a − d[0].a, d[1].b − d[0].b}
 for _, at := range w.gl[step] {
 ⟨Try this attachment on domino k 83⟩
 }
}

```

**82.** ⟨Keep the arrangement if the upper layer will do 82⟩ ≡

```

for _, p := range w.small {
 if w.deg[p] ≠ 2 {
 return
 }
}
if w.loose ∨ oneComponent(w.small, smallSteps) {
 w.found = append(w.found, append([]piece(Λ), w.sol...))
}

```

This code is used in section 81.

83.  $\langle$  Try this attachment on domino k 83  $\rangle \equiv$

```

if $w.used[at.piece]$ {
 continue
}
 $s0 := \mathbf{pt}\{d[0].a + at.small[0].a, d[0].b + at.small[0].b\}$
 $s1 := \mathbf{pt}\{d[0].a + at.small[1].a, d[0].b + at.small[1].b\}$
if $\neg, on := w.deg[s0]; on$ {
 continue
}
if $\neg, on := w.deg[s1]; on$ {
 continue
}
 $added, ok := w.place(s0, s1)$
if $ok \wedge w.promising(k+1)$ {
 $w.used[at.piece] = \mathbf{true}$
 $w.sol = \mathbf{append}(w.sol, \mathbf{piece}\{big: d, small: [2]\mathbf{pt}\{s0, s1\}\})$
 $w.run(k+1)$
 $w.sol = w.sol[:len(w.sol) - 1]$
 $w.used[at.piece] = \mathbf{false}$
}
 $w.unplace(s0, s1, added)$

```

This code is used in section 81.

84. The two small squares of a piece are diagonal neighbours, so the piece brings an edge of the upper layer's cycle with it. That edge has to be counted once, not twice—a mistake that makes every branch die at once, since both squares would reach degree two the moment they are put down.

$\langle$  Functions 5  $\rangle + \equiv$

```

func ($w * \mathbf{walk}$) $place(s0, s1 \mathbf{pt})([]\mathbf{edge}, \mathbf{bool})$ {
 $w.deg[s0], w.deg[s1] = 0, 0$
 $w.small = \mathbf{append}(w.small, s0, s1)$
 var $added []\mathbf{edge}$
 for $\neg, s := \mathbf{range} []\mathbf{pt} \{s0, s1\}$ {
 for $\neg, t := \mathbf{range} smallSteps$ {
 $n := \mathbf{pt}\{s.a + t.a, s.b + t.b\}$
 if $\neg, on := w.deg[n]; \neg on \vee (s \equiv s1 \wedge n \equiv s0)$ {
 continue
 }
 $added = \mathbf{append}(added, \mathbf{edge}\{s, n\})$
 }
 }
}
 $\langle$ Add the edges and see whether anything is crowded 85 $\rangle$
}

```



85.  $\langle \text{Add the edges and see whether anything is crowded } 85 \rangle \equiv$

```

ok := true
for _, e := range added {
 w.deg[e.u]++
 w.deg[e.v]++
 w.edges++
 if w.deg[e.u] > 2 $\vee$ w.deg[e.v] > 2 {
 ok = false
 }
}
return added, ok

```

This code is used in section 84.

86.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func (w *walk) unplace(s0, s1 pt, added []edge) {
 for _, e := range added {
 w.deg[e.u]--
 w.deg[e.v]--
 w.edges--
 }
 delete(w.deg, s0)
 delete(w.deg, s1)
 w.small = w.small[:len(w.small) - 2]
}

```

87.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func (w *walk) promising(next int) bool {
 if w.edges $\geq$ len(w.small) $\wedge$ len(w.small) < 2 * len(w.dom) {
 return false
 }
 left := w.reach[next]
 for _, s := range w.small {
 if w.deg[s] $\equiv$ 2 {
 continue
 }
 $\langle \text{Give up unless a later domino can still reach a neighbour of s } 88 \rangle$
 }
 return true
}

```

88. It is the *neighbouring* squares that must still be free, not the square itself: the square is already down, so of course no later domino can put anything there.

⟨ Give up unless a later domino can still reach a neighbour of s 88 ⟩ ≡

```

room := false
for _, t := range smallSteps {
 n := pt{s.a + t.a, s.b + t.b}
 if _, on := w.deg[n]; on ∨ ¬left[n] {
 continue
 }
 room = true
 break
}
if ¬room {
 return false
}

```

This code is used in section 87.

89. ⟨ Functions 5 ⟩ +≡

```

func solveCycle(cs []pt, gl map[pt][]attach, loose bool) [][]piece {
 var out [][]piece
 for _, m := range matchings(cs) {
 ⟨ Work out what the dominoes from k onwards can still touch 90 ⟩
 w := &walk{dom: m, gl: gl, reach: reach, deg: map[pt]int{ }, loose: loose}
 w.run(0)
 out = append(out, w.found...)
 }
 return out
}

```

90. ⟨ Work out what the dominoes from k onwards can still touch 90 ⟩ ≡

```

reach := make([]map[pt]bool, len(m) + 1)
reach[len(m)] = map[pt]bool{ }
for k := len(m) - 1; k ≥ 0; k-- {
 r := map[pt]bool{ }
 for p := range reach[k + 1] {
 r[p] = true
 }
 step := pt{m[k][1].a - m[k][0].a, m[k][1].b - m[k][0].b}
 for _, at := range gl[step] {
 for _, s := range at.small {
 r[pt{m[k][0].a + s.a, m[k][0].b + s.b}] = true
 }
 }
 reach[k] = r
}

```

This code is used in section 89.

**91. What answer 306 counts.** Every snake-in-the-box cycle of twenty large squares fits in a box of size  $3 \times 9$ ,  $4 \times 8$ ,  $5 \times 7$  or  $6 \times 6$ , and answer 306 reports  $(0, 0, 4 \cdot 9, 8 \cdot 8)$  solutions in those four cases. Those are what Algorithm M finds, and Algorithm M is not told about connectivity: six of the eight  $6 \times 6$  classes are spurious, their small squares making an 8-cycle and a 12-cycle rather than one cycle of twenty. So the counts have to be taken two ways—as answer 306’s model would take them, and with connectivity insisted on.

Answer 306’s model forbids two things this program’s search does not. Its multiplicity items rule out a 4-cycle. And its items exist only for  $0 < x < 2n$ ,  $0 < y < 2m$ , so the upper layer may not stick out of the box. Both have to be imposed to reproduce its numbers.

⟨Do exercise 306 91⟩  $\equiv$

```

gl := glue()
genuine := 0
for _, x := range []box { {3, 9}, {4, 8}, {5, 7}, {6, 6} } {
 t0 := time.Now()
 cs := snakeCycles(x, 20)
 ⟨Pave every cycle and sort the arrangements out 92⟩
 genuine += classesOf(tight, x.group())
 ⟨Report this box size 100⟩
}
fmt.Printf("essentially_different_solutions_altogether: %d\n", genuine)

```

This code is used in section 3.

**92.** ⟨Pave every cycle and sort the arrangements out 92⟩  $\equiv$

```

var loose, tight, asKnuth [][]piece
for _, c := range cs {
 for _, sol := range solveCycle(c, gl, true) {
 loose = append(loose, sol)
 if oneComponent(smalls(sol), smallSteps) {
 tight = append(tight, sol)
 }
 if ¬hasFourCycle(sol) ∧ ¬sticksOut(sol, x) {
 asKnuth = append(asKnuth, sol)
 }
 }
}

```

This code is used in section 91.

**93.** ⟨Functions 5⟩  $+\equiv$

```

func hasFourCycle(sol []piece) bool {
 for _, n := range cycleSizes(smalls(sol), smallSteps) {
 if n ≡ 4 {
 return true
 }
 }
 return false
}

```

94.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func sticksOut(sol []pt, x box) bool {
 for _, p := range smalls(sol) {
 if outside(p, x) {
 return true
 }
 }
 return false
}

```

95.  $\langle \text{Functions } 5 \rangle + \equiv$

```

func cycleSizes(ps []pt, steps []pt) []int {
 set := map[pt]bool{ }
 for _, p := range ps {
 set[p] = true
 }
 seen := map[pt]bool{ }
 var out []int
 for _, p := range ps {
 if seen[p] {
 continue
 }
 $\langle \text{Measure the component containing p } 96 \rangle$
 }
 sort.Ints(out)
 return out
}

```

96.  $\langle \text{Measure the component containing p } 96 \rangle \equiv$

```

n, queue := 0, []pt{p}
seen[p] = true
for len(queue) > 0 {
 q := queue[0]
 queue = queue[1:]
 n++
 for _, t := range steps {
 if r := (pt{q.a + t.a, q.b + t.b}); set[r] $\wedge$ $\neg$ seen[r] {
 seen[r] = true
 queue = append(queue, r)
 }
 }
}
out = append(out, n)

```

This code is used in section 95.

97.  $\langle \text{Functions 5} \rangle + \equiv$ 

```

func classesOf(sols [][]piece, g []xform) int {
 seen := map[string]bool{ }
 n := 0
 for _, s := range sols {
 if seen[skey(s)] {
 continue
 }
 n ++
 for _, t := range g {
 var m []piece
 for _, q := range s {
 m = append(m, t.on(q))
 }
 seen[skey(m)] = true
 }
 }
 return n
}

```

98. Answer 306 also remarks in passing that two of the large squares may touch at a corner, and that five of the eleven solutions do not have this “defect”. Every induced cycle turns corners, and at every turn two of its cells touch diagonally; what is meant must be a corner contact between cells that are *not* two steps apart round the cycle, where the ring doubles back and pinches itself.

 $\langle \text{Functions 5} \rangle + \equiv$ 

```

func pinches(sol []piece) bool {
 ring := ringOf(bigs(sol), bigSteps)
 pos := map[pt]int{ }
 for i, p := range ring {
 pos[p] = i
 }
 for i, p := range ring {
 $\langle \text{Look for a corner contact from far round the ring 99} \rangle$
 }
 return false
}

```

99.  $\langle$  Look for a corner contact from far round the ring 99  $\rangle \equiv$

```

for _, d := range []pt {{2, 2}, {2, -2}} {
 j, on := pos[pt{p.a + d.a, p.b + d.b}]
 if !on {
 continue
 }
 gap := i - j
 if gap < 0 {
 gap = -gap
 }
 if gap > len(ring) - gap {
 gap = len(ring) - gap
 }
 if gap > 2 {
 return true
 }
}

```

This code is used in section 98.

100.  $\langle$  Report this box size 100  $\rangle \equiv$

```

g := x.group()
fmt.Printf("%dx%d: %d_snake-in-the-box_20-cycles_fill_the_box\n", x.m, x.n, len(cs))
fmt.Printf("as_answer_306_counts_them_(no_4-cycle_upper_layer_inside):" +
 "%d_arrangements, %d_classes\n", len(asKnuth), classesOf(asKnuth, g))
 $\langle$ Say which of those are spurious 101 $\rangle$
fmt.Printf("one_cycle_in_both_layers: %d_arrangements, %d_classes (%s)\n",
 len(tight), classesOf(tight, g), time.Since(t0).Round(time.Millisecond))
 $\langle$ Say how many of the genuine ones pinch 102 $\rangle$

```

This code is used in section 91.

101.  $\langle$  Say which of those are spurious 101  $\rangle \equiv$

```

bad := map[string][][]piece{}
for _, sol := range asKnuth {
 if oneComponent(smalls(sol), smallSteps) {
 continue
 }
 k := fmt.Sprintf(cycleSizes(smalls(sol), smallSteps))
 bad[k] = append(bad[k], sol)
}
for k, l := range bad {
 fmt.Printf("of which %d_arrangements, %d_classes are spurious:" +
 "%d the upper layer is %s\n", len(l), classesOf(l, g), k)
}

```

This code is used in section 100.

**102.**  $\langle$  Say how many of the genuine ones pinch 102  $\rangle \equiv$

```

var yes, no [[]]piece
for _, sol := range tight {
 if pinches(sol) {
 yes = append(yes, sol)
 } else {
 no = append(no, sol)
 }
}
if len(tight) > 0 {
 fmt.Printf(" %d classes pinch, %d do not\n",
 classesOf(yes, g), classesOf(no, g))
}

```

This code is used in section 100.

**103. Index.**

- a*: [4](#), [5](#), [6](#), [8](#), [10](#), [12](#), [14](#), [17](#), [23](#), [24](#), [25](#), [29](#), [31](#), [34](#),  
[40](#), [47](#), [50](#), [53](#), [54](#), [67](#), [68](#), [69](#), [74](#), [75](#), [78](#), [81](#), [83](#),  
[84](#), [88](#), [90](#), [96](#), [99](#).  
*add*: [45](#), [61](#).  
*added*: [83](#), [84](#), [85](#), [86](#).  
*ahead*: [11](#), [12](#), [14](#), [27](#), [43](#), [73](#), [76](#).  
*all*: [32](#), [35](#), [42](#), [57](#), [58](#).  
*append*: [16](#), [29](#), [33](#), [34](#), [36](#), [37](#), [38](#), [40](#), [42](#), [43](#), [49](#),  
[58](#), [60](#), [64](#), [68](#), [69](#), [71](#), [72](#), [73](#), [74](#), [75](#), [76](#), [78](#), [82](#),  
[83](#), [84](#), [89](#), [92](#), [96](#), [97](#), [101](#), [102](#).  
*asKnuth*: [92](#), [100](#), [101](#).  
*at*: [81](#), [83](#), [90](#).  
*Atoi*: [25](#).  
**attach**: [77](#), [78](#), [79](#), [89](#).  
*b*: [4](#), [5](#), [6](#), [8](#), [10](#), [12](#), [14](#), [17](#), [23](#), [24](#), [25](#), [29](#), [31](#), [32](#),  
[34](#), [35](#), [40](#), [43](#), [47](#), [50](#), [53](#), [54](#), [67](#), [69](#), [70](#), [74](#), [75](#),  
[78](#), [81](#), [83](#), [84](#), [88](#), [90](#), [96](#), [99](#).  
*bad*: [101](#).  
*best*: [15](#).  
*big*: [7](#), [9](#), [10](#), [11](#), [13](#), [16](#), [17](#), [20](#), [29](#), [35](#), [36](#), [37](#), [41](#),  
[43](#), [78](#), [83](#).  
*bigFrame*: [59](#), [61](#), [62](#).  
*bigMirror*: [59](#), [61](#), [62](#).  
*big*s: [37](#), [53](#), [61](#), [63](#), [98](#).  
*bigSteps*: [19](#), [69](#), [72](#), [76](#), [98](#).  
*bigSym*: [59](#), [61](#), [62](#).  
*bk*: [61](#).  
*Bool*: [2](#).  
*bool*: [5](#), [6](#), [12](#), [14](#), [17](#), [21](#), [23](#), [26](#), [28](#), [31](#), [42](#), [43](#), [47](#),  
[48](#), [51](#), [52](#), [53](#), [54](#), [56](#), [57](#), [73](#), [74](#), [75](#), [79](#), [84](#), [87](#),  
[89](#), [90](#), [93](#), [94](#), [95](#), [97](#), [98](#).  
**box**: [22](#), [23](#), [26](#), [31](#), [40](#), [53](#), [55](#), [56](#), [67](#), [71](#), [91](#), [94](#).  
*build*: [32](#), [57](#), [63](#).  
*Builder*: [32](#), [43](#), [67](#).  
*c*: [14](#), [43](#), [54](#), [67](#), [68](#), [69](#), [92](#).  
*canon*: [15](#), [17](#), [21](#).  
*caps*: [69](#), [70](#).  
*cells*: [1](#), [10](#), [13](#), [14](#), [36](#), [59](#), [63](#), [71](#).  
*classes*: [46](#), [62](#).  
*classesOf*: [91](#), [97](#), [100](#), [101](#), [102](#).  
*col*: [70](#).  
*Contains*: [61](#).  
*count*: [63](#), [64](#).  
*cs*: [89](#), [91](#), [92](#), [100](#).  
*cur*: [75](#).  
*cycleModel*: [67](#), [71](#).  
*cycleSizes*: [93](#), [95](#), [101](#).  
*d*: [17](#), [69](#), [74](#), [75](#), [78](#), [81](#), [83](#), [99](#).  
*da*: [10](#), [28](#), [29](#).  
*Dance*: [59](#), [63](#), [71](#).  
*db*: [10](#), [28](#), [29](#).  
*deg*: [79](#), [82](#), [83](#), [84](#), [85](#), [86](#), [87](#), [88](#), [89](#).  
*delete*: [86](#).  
*dom*: [79](#), [81](#), [87](#), [89](#).  
*e*: [85](#), [86](#).  
**edge**: [80](#), [84](#), [86](#).  
*edges*: [79](#), [85](#), [86](#), [87](#).  
*el*: [59](#), [62](#).  
*f*: [39](#), [41](#), [47](#), [49](#), [51](#), [62](#).  
*fitting*: [59](#), [61](#), [62](#).  
*fix*: [45](#), [60](#), [61](#).  
*flag*: [2](#).  
*fmt*: [14](#), [20](#), [21](#), [24](#), [33](#), [35](#), [43](#), [58](#), [62](#), [64](#), [68](#), [69](#),  
[70](#), [91](#), [100](#), [101](#), [102](#).  
*found*: [79](#), [82](#), [89](#).  
*Fprintf*: [35](#), [43](#), [69](#), [70](#).  
*frameSym*: [51](#), [61](#).  
*G*: [44](#).  
*g*: [17](#), [40](#), [46](#), [57](#), [62](#), [63](#), [65](#), [97](#), [100](#), [101](#), [102](#).  
*gap*: [99](#).  
*genuine*: [91](#).  
*gl*: [79](#), [81](#), [89](#), [90](#), [91](#), [92](#).  
*glue*: [78](#), [91](#).  
*got*: [48](#), [49](#).  
*grid*: [53](#), [63](#).  
*group*: [40](#), [42](#), [51](#), [91](#), [100](#).  
*ha*: [47](#), [48](#), [49](#).  
*hasFourCycle*: [92](#), [93](#).  
*HasPrefix*: [71](#).  
*HasSuffix*: [71](#).  
*hb*: [47](#), [48](#), [49](#).  
*hiA*: [48](#), [49](#), [50](#), [52](#), [72](#).  
*hiB*: [48](#), [49](#), [50](#), [52](#), [72](#).  
*i*: [9](#), [10](#), [14](#), [15](#), [20](#), [25](#), [29](#), [32](#), [33](#), [35](#), [41](#), [43](#), [54](#),  
[58](#), [70](#), [73](#), [78](#), [98](#), [99](#).  
*in*: [57](#), [59](#), [63](#).  
*IndexByte*: [25](#).  
*insideLarge*: [23](#), [29](#), [69](#).  
*int*: [4](#), [14](#), [22](#), [43](#), [44](#), [45](#), [46](#), [47](#), [50](#), [52](#), [54](#), [59](#), [63](#),  
[67](#), [71](#), [73](#), [77](#), [79](#), [81](#), [87](#), [89](#), [95](#), [97](#), [98](#).  
*Ints*: [95](#).  
*j*: [14](#), [43](#), [54](#), [70](#), [73](#), [99](#).  
*Join*: [32](#), [48](#), [65](#), [67](#).  
*k*: [15](#), [16](#), [21](#), [60](#), [62](#), [64](#), [70](#), [76](#), [81](#), [83](#), [90](#), [101](#).



- key*: [14](#), [15](#).  
*keys*: [16](#), [21](#), [64](#).  
*kind*: [63](#), [64](#).  
*known*: [21](#).  
*l*: [42](#), [58](#), [101](#).  
*large*: [5](#), [23](#), [34](#), [36](#).  
*largePrim*: [26](#), [34](#), [55](#).  
*left*: [87](#), [88](#).  
*legal*: [42](#).  
*len*: [20](#), [21](#), [32](#), [57](#), [58](#), [64](#), [65](#), [70](#), [71](#), [72](#), [74](#), [75](#), [76](#),  
[81](#), [83](#), [86](#), [87](#), [90](#), [96](#), [99](#), [100](#), [101](#), [102](#).  
*lines*: [65](#).  
*loA*: [48](#), [49](#), [50](#), [52](#), [72](#).  
*loB*: [48](#), [49](#), [50](#), [52](#), [72](#).  
*loose*: [79](#), [82](#), [89](#), [92](#).  
*m*: [22](#), [23](#), [28](#), [31](#), [34](#), [40](#), [52](#), [53](#), [58](#), [67](#), [68](#), [69](#), [72](#),  
[76](#), [89](#), [90](#), [97](#), [100](#).  
*main*: [1](#).  
*make*: [32](#), [90](#).  
*matchings*: [76](#), [89](#).  
*mc*: [71](#).  
*Millisecond*: [59](#), [100](#).  
*minA*: [10](#).  
*minB*: [10](#).  
*mode*: [2](#), [3](#).  
*moved*: [60](#), [75](#).  
*n*: [22](#), [23](#), [28](#), [31](#), [34](#), [40](#), [52](#), [53](#), [58](#), [67](#), [68](#), [69](#), [72](#),  
[84](#), [88](#), [93](#), [96](#), [97](#), [100](#).  
*name*: [39](#), [47](#), [49](#), [58](#).  
*names*: [58](#).  
*nb*: [36](#), [69](#), [70](#).  
*NewMCC*: [71](#).  
*NewReader*: [59](#), [63](#), [71](#).  
*NewXCC*: [59](#), [63](#).  
*next*: [87](#).  
*no*: [102](#).  
*Nodes*: [62](#).  
*nopt*: [58](#).  
*Now*: [59](#), [91](#).  
*ns*: [36](#).  
*o*: [36](#), [71](#).  
*off*: [76](#).  
*ok*: [42](#), [49](#), [51](#), [83](#), [85](#).  
*on*: [41](#), [42](#), [53](#), [60](#), [71](#), [72](#), [83](#), [84](#), [88](#), [97](#), [99](#).  
*oneComponent*: [72](#), [74](#), [82](#), [92](#), [101](#).  
*opt*: [36](#).  
*Option*: [36](#).  
*options*: [28](#), [32](#).  
*out*: [16](#), [28](#), [29](#), [36](#), [37](#), [38](#), [42](#), [54](#), [71](#), [72](#), [73](#), [75](#),  
[76](#), [78](#), [89](#), [95](#), [96](#).  
*outside*: [30](#), [31](#), [94](#).  
*p*: [5](#), [6](#), [8](#), [10](#), [14](#), [17](#), [23](#), [24](#), [31](#), [34](#), [36](#), [40](#), [47](#), [48](#),  
[49](#), [50](#), [51](#), [53](#), [69](#), [70](#), [73](#), [74](#), [75](#), [82](#), [90](#), [94](#), [95](#),  
[96](#), [98](#), [99](#).  
*pairKey*: [27](#), [30](#), [56](#).  
*pairs*: [26](#), [30](#), [56](#).  
*panic*: [55](#).  
*Parse*: [2](#).  
*parse*: [36](#), [59](#), [63](#).  
*part*: [55](#), [57](#), [63](#).  
*pc*: [17](#).  
*piece*: [7](#), [9](#), [10](#), [11](#), [13](#), [14](#), [15](#), [16](#), [17](#), [21](#), [28](#), [29](#),  
[32](#), [36](#), [37](#), [38](#), [41](#), [42](#), [43](#), [53](#), [60](#), [77](#), [79](#), [82](#), [83](#),  
[89](#), [92](#), [93](#), [94](#), [97](#), [98](#), [101](#), [102](#).  
*pieces*: [16](#), [20](#), [32](#), [78](#).  
*piecesPrim*: [26](#), [33](#), [55](#).  
*pinches*: [98](#), [102](#).  
*pl*: [21](#).  
*place*: [83](#), [84](#).  
*pos*: [98](#), [99](#).  
*prim*: [32](#), [33](#), [34](#), [67](#), [68](#).  
*printed*: [21](#).  
*Printf*: [20](#), [21](#), [58](#), [62](#), [64](#), [91](#), [100](#), [101](#), [102](#).  
*prob*: [26](#), [28](#), [32](#), [42](#), [51](#), [55](#), [56](#).  
*promising*: [83](#), [87](#).  
*ps*: [20](#), [21](#), [32](#), [35](#), [48](#), [50](#), [51](#), [52](#), [73](#), [74](#), [75](#), [76](#), [95](#).  
*pt*: [4](#), [5](#), [6](#), [7](#), [8](#), [10](#), [12](#), [13](#), [16](#), [17](#), [18](#), [19](#), [21](#), [23](#),  
[24](#), [25](#), [26](#), [27](#), [29](#), [31](#), [34](#), [37](#), [38](#), [39](#), [40](#), [47](#), [48](#),  
[50](#), [51](#), [52](#), [53](#), [54](#), [55](#), [56](#), [68](#), [69](#), [71](#), [73](#), [74](#), [75](#),  
[76](#), [77](#), [78](#), [79](#), [80](#), [81](#), [83](#), [84](#), [86](#), [88](#), [89](#), [90](#), [95](#),  
[96](#), [98](#), [99](#).  
*q*: [9](#), [10](#), [11](#), [13](#), [14](#), [15](#), [17](#), [20](#), [21](#), [28](#), [36](#), [37](#), [38](#),  
[41](#), [42](#), [43](#), [56](#), [60](#), [69](#), [70](#), [74](#), [75](#), [78](#), [96](#), [97](#).  
*queue*: [74](#), [96](#).  
*r*: [9](#), [15](#), [28](#), [29](#), [41](#), [78](#), [90](#), [96](#).  
*reach*: [79](#), [87](#), [89](#), [90](#).  
*rect*: [54](#), [55](#).  
*report*: [3](#), [57](#).  
*ring*: [76](#), [98](#), [99](#).  
*ringOf*: [75](#), [76](#), [98](#).  
*room*: [88](#).  
*rot*: [8](#), [9](#).  
*Round*: [59](#), [100](#).  
*run*: [81](#), [83](#), [89](#).  
*s*: [14](#), [25](#), [29](#), [30](#), [35](#), [53](#), [59](#), [63](#), [71](#), [78](#), [84](#), [87](#), [88](#),  
[90](#), [97](#).  
*s0*: [83](#), [84](#), [86](#).

- s1*: [83](#), [84](#), [86](#).  
*sec*: [32](#), [33](#), [34](#), [67](#), [68](#).  
*seen*: [16](#), [17](#), [28](#), [29](#), [73](#), [74](#), [75](#), [95](#), [96](#), [97](#).  
*set*: [48](#), [49](#), [51](#), [74](#), [75](#), [95](#), [96](#).  
*shapeKind*: [48](#), [61](#), [63](#).  
*shapeMaps*: [47](#), [48](#).  
*shapes*: [2](#), [59](#), [61](#), [62](#).  
*Since*: [59](#), [100](#).  
*skey*: [43](#), [60](#), [97](#).  
*Slice*: [14](#), [43](#), [73](#).  
*small*: [6](#), [7](#), [9](#), [10](#), [11](#), [13](#), [17](#), [20](#), [29](#), [30](#), [34](#), [35](#), [36](#),  
[38](#), [41](#), [43](#), [77](#), [78](#), [79](#), [82](#), [83](#), [84](#), [86](#), [87](#), [90](#).  
*smallAsym*: [59](#), [61](#), [62](#).  
*smallPrim*: [26](#), [30](#), [34](#), [55](#), [56](#).  
*smalls*: [38](#), [61](#), [92](#), [93](#), [94](#), [101](#).  
*smallSteps*: [17](#), [18](#), [82](#), [84](#), [88](#), [92](#), [93](#), [101](#).  
*smallSym*: [59](#), [61](#), [62](#).  
*snake*: [2](#).  
*snakeCycles*: [71](#), [91](#).  
*sol*: [37](#), [38](#), [43](#), [53](#), [59](#), [60](#), [61](#), [63](#), [79](#), [82](#), [83](#), [92](#), [93](#),  
[94](#), [98](#), [101](#), [102](#).  
*sols*: [44](#), [45](#), [62](#), [97](#).  
*Solutions*: [59](#), [63](#), [71](#).  
*solveCycle*: [89](#), [92](#).  
*sort*: [14](#), [16](#), [43](#), [64](#), [73](#), [95](#).  
*sorted*: [10](#), [11](#), [29](#), [36](#), [41](#), [78](#).  
*span*: [48](#), [50](#), [52](#), [72](#).  
*spans*: [52](#), [61](#).  
*Split*: [65](#).  
*Sprint*: [101](#).  
*Sprintf*: [14](#), [24](#), [33](#), [68](#).  
*sq*: [24](#), [34](#), [35](#), [47](#), [49](#), [68](#), [69](#), [70](#).  
*stab*: [59](#), [60](#), [62](#).  
*step*: [81](#), [90](#).  
*steps*: [74](#), [75](#), [95](#), [96](#).  
*sticksOut*: [92](#), [94](#).  
*strconv*: [25](#).  
*String*: [2](#), [32](#), [43](#), [67](#).  
*string*: [14](#), [15](#), [16](#), [21](#), [24](#), [25](#), [32](#), [39](#), [43](#), [47](#), [48](#), [53](#),  
[55](#), [57](#), [58](#), [59](#), [63](#), [64](#), [65](#), [67](#), [69](#), [97](#), [101](#).  
*Strings*: [16](#), [64](#).  
*strings*: [25](#), [32](#), [43](#), [48](#), [59](#), [61](#), [63](#), [65](#), [67](#), [71](#).  
*sym*: [57](#), [60](#).  
*sym0*: [58](#).  
*symmetries*: [42](#), [57](#), [58](#).  
*t*: [36](#), [41](#), [42](#), [45](#), [46](#), [48](#), [49](#), [51](#), [58](#), [60](#), [71](#), [84](#), [88](#),  
[96](#), [97](#).  
*t0*: [59](#), [91](#), [100](#).  
**tally**: [44](#), [45](#), [46](#), [59](#).  
*tidy*: [9](#), [10](#), [15](#), [17](#), [28](#), [78](#).  
*tight*: [91](#), [92](#), [100](#), [102](#).  
*time*: [59](#), [91](#), [100](#).  
*total*: [59](#), [62](#).  
*touching*: [17](#).  
*Trim*: [65](#).  
*trim*: [63](#), [65](#).  
*TrimRight*: [65](#).  
*turn*: [9](#), [15](#), [28](#), [78](#).  
*u*: [12](#), [27](#), [54](#), [76](#), [80](#), [85](#), [86](#).  
*unique*: [72](#), [73](#).  
*unplace*: [83](#), [86](#).  
*unsq*: [25](#), [36](#), [71](#).  
*used*: [79](#), [83](#).  
*v*: [12](#), [27](#), [54](#), [62](#), [76](#), [80](#), [85](#), [86](#).  
*verbose*: [2](#), [3](#), [57](#), [58](#).  
*w*: [28](#), [29](#), [30](#), [32](#), [33](#), [34](#), [42](#), [51](#), [56](#), [57](#), [58](#), [61](#), [63](#),  
[81](#), [82](#), [83](#), [84](#), [85](#), [86](#), [87](#), [88](#), [89](#).  
**walk**: [79](#), [81](#), [84](#), [86](#), [87](#), [89](#).  
*want*: [67](#), [68](#), [71](#), [72](#).  
*weight*: [44](#), [45](#), [46](#).  
*which*: [55](#), [57](#), [58](#), [62](#).  
*WriteByte*: [32](#).  
*WriteString*: [32](#), [67](#), [70](#).  
*x*: [23](#), [26](#), [28](#), [29](#), [30](#), [31](#), [34](#), [40](#), [42](#), [51](#), [53](#), [55](#), [56](#),  
[58](#), [63](#), [67](#), [68](#), [69](#), [71](#), [72](#), [91](#), [92](#), [94](#), [100](#).  
*xc*: [59](#), [62](#), [63](#).  
**xform**: [39](#), [40](#), [41](#), [42](#), [97](#).  
*yes*: [102](#).

- ⟨Add the edges and see whether anything is crowded 85⟩ Used in section 84
- ⟨Ask whether the upper layer allows this placement 30⟩ Used in section 29
- ⟨Check the piece set 20⟩ Used in section 3
- ⟨Compare with the placements answer 305 prints 21⟩ Used in section 20
- ⟨Count the solutions and sort them by symmetry 59⟩ Used in section 57
- ⟨Declarations 4⟩ Used in section 1
- ⟨Declare the item for piece i 33⟩ Used in section 32
- ⟨Declare the items for the squares 34⟩ Used in section 32
- ⟨Do exercise 306 91⟩ Used in section 3
- ⟨Do what the mode asks 3⟩ Used in section 1
- ⟨Functions 5⟩ Used in section 1
- ⟨Give up unless a later domino can still reach a neighbour of s 88⟩ Used in section 87
- ⟨Glue every small domino onto this large one 17⟩ Used in section 16
- ⟨Keep the arrangement if the upper layer will do 82⟩ Used in section 81
- ⟨Keep this one if it is a single cycle filling the box 72⟩ Used in section 71
- ⟨List the shapes the large squares make 63⟩ Used in section 3
- ⟨Look for a corner contact from far round the ring 99⟩ Used in section 98
- ⟨Measure the component containing p 96⟩ Used in section 95
- ⟨Name the items of the cycle problem 68⟩ Used in section 67
- ⟨Pave every cycle and sort the arrangements out 92⟩ Used in section 91
- ⟨Print the shapes that are symmetric 64⟩ Used in section 63
- ⟨Read the command line 2⟩ Used in section 1
- ⟨Report this box size 100⟩ Used in section 91
- ⟨Say how big the problem is 58⟩ Used in section 57
- ⟨Say how many of the genuine ones pinch 102⟩ Used in section 100
- ⟨Say what the counts came to 62⟩ Used in section 57
- ⟨Say which of those are spurious 101⟩ Used in section 100
- ⟨Set up the windmill of part (a) 56⟩ Used in section 55
- ⟨Slide the piece there and keep it if it fits 29⟩ Used in section 28
- ⟨Tally what its two layers look like 61⟩ Used in section 59
- ⟨Try this attachment on domino k 83⟩ Used in section 81
- ⟨Try this map on the shape 49⟩ Used in section 48
- ⟨Work out the stabilizer of this arrangement 60⟩ Used in section 59
- ⟨Work out what the dominoes from k onwards can still touch 90⟩ Used in section 89
- ⟨Write one option per choice of two neighbours 70⟩ Used in section 69
- ⟨Write one option per placement 35⟩ Used in section 32
- ⟨Write the options for cell (a,c) 69⟩ Used in section 67

# Windmill Dominoes

|                                      | Section             | Page |
|--------------------------------------|---------------------|------|
| Introduction .....                   | <a href="#">1</a>   | 1    |
| The windmill dominoes .....          | <a href="#">4</a>   | 3    |
| Boxes and layers .....               | <a href="#">22</a>  | 8    |
| The exact cover problem .....        | <a href="#">32</a>  | 11   |
| Symmetries of a box .....            | <a href="#">39</a>  | 14   |
| The shape of a layer .....           | <a href="#">47</a>  | 17   |
| The five parts of exercise 305 ..... | <a href="#">54</a>  | 20   |
| Two snakes at once .....             | <a href="#">66</a>  | 25   |
| Perfect matchings .....              | <a href="#">75</a>  | 29   |
| Walking the dominoes .....           | <a href="#">79</a>  | 31   |
| What answer 306 counts .....         | <a href="#">91</a>  | 35   |
| Index .....                          | <a href="#">103</a> | 40   |